

Sofia University
Department of Mathematics and Informatics

Course : OO Programming C#.NET

Date: May 29, 2025

Student Name:

Lab No. 13b

Submit the all C# .NET files developed to solve the problems listed below. Use comments and Modified-Hungarian notation.

Problem No. 1

Complete tutorial No. 1

Problem No.2

Complete tutorial No. 2

Problem No.3

Complete tutorial No. 3

Problem No.4

Complete tutorial Multi-threading with the BackgroundWorker

Problem No. 5

Write a Console application to check the status of a list of websites status using threads from the ThreadPool.

Problem No. 6a

Write a method

```
void ShowFileContents(string filename)
```

to read a text file asynchronously.

Problem No. 6b

Write a method

```
async Task ShowFileContents(string filename)
```

to read a text file asynchronously.

Problem No. 7

Write an application n to run two thread in a ThreadPool. The first thread prints the string passed to it as a parameter, while the second thread prints the square of the integer passed to it as a parameter. Use ManualResetEvent to wait for completion of thread execution.

Problem No. 8

Sometimes, we may want to queue a work item in the ThreadPool that returns a result. For this, we can use the Task class, which represents an asynchronous operation that can return a value. The Task class provides a static method called Run that allows us to queue a work item (a delegate or a lambda expression) to the ThreadPool and return a Task object that represents the operation. The Task object

has a property called `Result` that returns the value of the work item when it completes. For example, the following code queues two work items that return the sum and the product of two numbers. Write an app that queues two work items on the `ThreadPool` so that they return the sum and the product of two numbers.

Problem No.9

Run the following app

`class Program`

```
// A shared variable to count the number of work items completed
static int counter = 0;

static void Main(string[] args)
{
    // Queue 10 work items to the ThreadPool
    for (int i = 0; i < 10; i++)
    {
        ThreadPool.QueueUserWorkItem(WorkItem);
    }

    // Wait for the work items to complete
    Thread.Sleep(1000);

    Console.WriteLine("Counter: {0}", counter);

    Console.WriteLine("Main thread exits");
}

// A work item that increments the counter
static void WorkItem(object state)
{
    // Increment the counter
    counter++;
    Thread.Sleep(10);
    Console.WriteLine("WorkItem: {0}", counter);
}
}
```

Note, that the output of the program may vary depending on the order of execution of the work items, but it could look something like this

```
WorkItem: 8
WorkItem: 8
WorkItem: 8
WorkItem: 8
WorkItem: 9
WorkItem: 10
WorkItem: 8
WorkItem: 9
WorkItem: 9
WorkItem: 8
Counter: 10
Main thread exits
```

Correct the app by use a synchronization mechanism such as a `lock` statement or an `Interlocked` method to ensure that only one thread can access the shared variable at a time.